MosaicOpt: An Algebraic Foundation for Agentic Memory Databases

ANONYMOUS AUTHOR(S)

Production LLM agents accumulate multi-modal memory across two timescales: long-term facts and preferences, and short-term working context. A memory unit typically carries text, one or more dense vector embeddings, structured metadata, and graph relations. Useful queries against this store are conjunctive expressions that mix all four modalities — for example, "find passages about a fault-tolerance bug from the last 30 days, semantically similar to a stack trace, attributed to a known subsystem, and connected by a derivation link to a prior incident." The dominant engineering response, *multi-path recall* with application-level score fusion, has no principled semantics, no equivalence relation over plans, and breaks down on multi-vector conjunctions. We propose **MosaicOpt**, an *algebra-first* design that extends relational algebra with (i) a *materialization* operator μ that binds computed values (vector distances, BM25 scores) as first-class attributes, (ii) first-class vector, text, and graph predicates, and (iii) a graph expansion operator Γ. The optimizer is a Volcano/Cascades-style rule-based layer driven by an *equational theory* of seven identities (M1–M3, V1, J1, T1, G1), combined with a cost-based layer that explicitly accounts for approximation. We prove the identities, give counterexamples for the non-identities (N1–N3) that classical intuition suggests but approximation breaks, and sketch a prototype and evaluation plan. The argument is foundational: agentic memory is a query problem, query problems benefit from algebraic foundations, and the right foundation is an extension of relational algebra.

Additional Key Words and Phrases: agentic memory, relational algebra, query optimization, vector search, hybrid retrieval, rule-based optimizer, equational theory

ACM Reference Format:
Anonymous Author(s). 2026. MosaicOpt: An Algebraic Foundation for Agentic Memory Databases. In *Proceedings of ACM SIGMOD/PODS International Conference on Management of Data / Principles of Database Systems (PODS '27).* ACM, New York, NY, USA, 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction & Motivation

Large language model (LLM) agents are no longer stateless request–response loops. Production agents — coding copilots, personal assistants, autonomous research workers, multi-step planners — accumulate state across two timescales: *long-term memory*, which persists facts, preferences, and episodic traces across sessions; and *short-term memory*, which captures the working context of an in-flight session, including tool-call history and intermediate reasoning artifacts. This split is widely accepted in the agentic-systems literature, but its consequences for the storage layer have received less attention.

The content of agentic memory is fundamentally *multi-modal.* A single "memory unit" may carry: (i) raw text (a chat snippet, a code diff, a document excerpt); (ii) one or more dense vector embeddings, often produced by different encoders for different retrieval objectives (e.g. semantic similarity, intent classification, code search); (iii) structured metadata (timestamps, source, provenance, user tags, confidence scores); and (iv) graph structure (entity relations such as `derived_from`, `cites`, `contradicts`, `member_of`). A useful query against this store is seldom a pure similarity lookup; it is a *conjunctive* expression that mixes all four modalities. "Find passages about a fault tolerance bug from the last 30 days, semantically similar to this stack trace, attributed to a known subsystem, and connected by `derived_from` to a previous incident" is representative.

PODS '27,
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

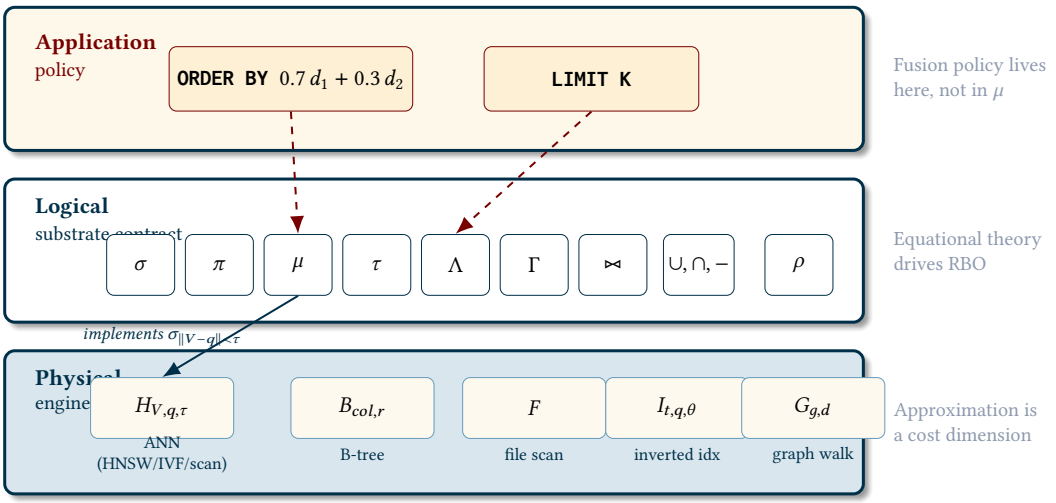


Fig. 1. Three-layer architecture of MosaicOpt: *application* (SQL `ORDER BY` and `LIMIT`, holding the fusion policy), *logical algebra* (the substrate contract including the new μ and Γ operators, over which the equational theory is stated), and *physical algebra* (the engine implementation). The vector selection $\sigma_{\|V-q\|<\tau}$ is realized by the $H_{V,q,\tau}$ physical operator, whose concrete realization is an *ANN family* (HNSW, IVF, or brute-force linear scan); Section 5 selects the strategy per query.

The dominant engineering response is *multi-path recall*: a fan-out across specialized engines (a vector index, a full-text index, a key–value store, a graph store), each producing a ranked candidate list, followed by application-level score fusion and re-ranking. Frameworks such as Mem0, Letta, Zep, and Cognee instantiate variations of this pattern. It works, and there is no shortage of impressive product demonstrations. But the pattern has four structural weaknesses:

(1) **No principled semantics.** The semantics of a fused query is defined by the application code that combines the per-channel scores. Two implementations of "find similar memory" in the same product can have silently different meanings.
(2) **Heuristic optimization only.** Without an algebra, there is no equivalence relation over plans. The "optimizer" is a hand-tuned orchestration pipeline whose rewrites are not derived from a sound theory.
(3) **Fusion policy leaks into the substrate.** Score combination functions (e.g. weighted sum, reciprocal-rank fusion, learned rerankers) are application concerns; once baked into a query template, they are hard to change without re-engineering the data path.
(4) **Top-k per column breaks conjunctive intent.** Each vector column is typically searched independently with its own top-k, then intersected. For multi-vector memories, the intersection of per-column top-k sets is frequently empty or impoverished, because the columns are correlated, not redundant.

Our position is that these problems are not implementation details; they are symptoms of a missing foundation. **MosaicOpt** proposes an *algebra-first* design in which relational algebra is extended with a small set of operators that make multi-modal memory queries first-class, and in which the resulting algebra — not application code — is the substrate contract between query and execution. The optimizer becomes a *rule-based* layer driven by an equational theory (Section 3), with a *cost-based* layer (Section 5) choosing between physical realizations that may legitimately differ in approximation behaviour.

Figure 1 shows the resulting architecture: application policy on top, logical algebra as the substrate contract, physical algebra as the engine layer. The ANN family bridge $H_{V,q,\tau}$ is the point at which approximation enters the system — a fact the logical layer hides and the cost model (Section 5) makes explicit.

1.1 Why algebraic foundation?

Codd's relational algebra [1] and Selinger's cost-based optimizer [2] established that query processing benefits from a formal language for what is computable and a formal theory for what transformations preserve semantics. The Volcano/Cascades line further showed that the algebra is the *API*: a new operator ships with its equivalence rules and cost model, and the optimizer gains new capabilities without any central rewrite. **An algebra is not an implementation detail; it is the contract between query and execution.** When the algebra is rigorous, every query language is a syntactic surface over it, every optimizer decision is auditable, and every physical innovation is a drop-in.

Formally, an algebra provides four things that no collection of indexes and APIs can:

- **A language for expressive power** — a precise definition of what is computable in the system, and what is not.
- **An equational theory** — a body of identities that govern when one query plan is equivalent to another.
- **A cost-model framework** — the substrate on which cost-based optimization rests, parameterized by the same logical operators the user wrote.
- **Composability** — new operators can be added without breaking old ones, because the algebra is the integration point. The Cascades maxim — *the algebra is the API* — makes extensibility a property of the foundation, not of the optimizer code.

Agentic memory is the *worst case* for a "bag of engines" approach. The reasons are structural:

- It spans two timescales (long-term and short-term) that demand different access patterns, lifetime semantics, and consistency guarantees.
- Its content is genuinely multi-modal: text, multiple vector columns, structured metadata, and graph relations all live on the same memory units.
- The queries that matter are *conjunctive across modalities* — a vector range intersected with a metadata filter joined with a graph reachability predicate. Ad hoc orchestration is exactly where it fails.
- There is *convergent evolution* in the field: every Agentic framework (Mem0, Letta, Zep, Cognee) has independently reinvented the same patterns (multi-path recall, score fusion, re-ranking) but with no shared vocabulary, no shared type system, and no shared notion of plan equivalence.
- The practical consequence: each framework has a different *memory API* with no formal semantics. The same SQL-like query may have silently different meanings across implementations, and porting a workload requires reading source code rather than reasoning from a specification.

1.2 Mathematical support for Agentic database development

The algebra gives database researchers and engineers a precise substrate on which to develop new query patterns and to *prove* their correctness. A new materialization pattern (e.g., a new kind of multi-modal fusion, a new join algorithm for hybrid vectors) can be expressed as a new operator with a documented equational theory — its rewrite rules are *derived* from the algebra, not guessed from benchmarks. The optimizer's plan space becomes a *derivable* structure: each new operator is added with its identities, and the optimizer gains the corresponding rewrites "for free". This is exactly the extensibility property of Cascades; the algebra is what makes it principled.

A database system built on an algebra exposes a *substrate capability*: any application that can express its query in the algebra can take advantage of the optimizer's rewrites, and any physical innovation that implements an algebraic operator improves every application that uses it. The algebra is the *common currency* between application and engine: an application written once is portable across engines that share the algebra, and an engine innovation is portable to every application that uses the affected operator.

1.3 Data substrate for Agentic applications

Four representative use cases illustrate the substrate requirements: **episodic recall** (conjunctive vector + metadata + graph queries, where a single memory unit is retrieved only if it satisfies a similarity predicate *and* a time window *and* a graph reachability test); **cross-session fact retrieval** (multi-vector recall with structured filters, e.g., a user-scope or a topic filter); **working context assembly** (fast filtered recall by recency and entity, where latency dominates); **tool-trace analysis** (graph queries over `derived_from` and `follows` edges, joined with vector similarity on the underlying tool outputs).

All four are *query problems* — expressible as expressions over a memory store and amenable to the same optimizer pipeline. With MosaicOpt's algebra, an application writes its memory query in SQL (Section 4) or in the algebra directly; the substrate applies M1-style rewrites to fuse materialization with selection, V1/J1/T1/G1 to push predicates down, and cost-based physical-operator selection. The application does not need to know about HNSW/IVF/scan, B-tree, or inverted indexes — these are hidden as physical realizations of a single logical operator. The application policy (fusion weights, thresholds, top-k) stays in the SQL; the substrate's responsibility (index choice, join order, approximation budget) is delegated to the optimizer.

1.4 Contributions

This paper makes the following contributions:

- **An extended relational algebra for agentic memory.** We add a *materialization* operator μ, first-class *vector*, *text*, and *graph* predicates, and an explicit *graph expansion* operator Γ. Bag semantics and a strict logical/physical separation complete the model (Section 2).
- **An equational theory with proofs.** We state and prove seven identities (M1–M3, V1, J1, T1, G1) and document the *non-identities* that classical intuition suggests but approximation breaks (N1–N3, Section 3.4). The RBO is provably confluent (Theorem 11).
- **A three-layer architecture.** Application policy (SQL `ORDER BY`, `LIMIT`) is separated from the logical algebra (substrate contract) and the physical algebra (engine implementation). The substrate exposes per-column distances; fusion is the application's job.
- **A sketch of the optimizer.** A Volcano/Cascades-style RBO that uses the equational theory to generate equivalent logical plans, and a CBO that picks physical realizations under a cost model that explicitly accounts for approximation (Sections 4.7–5).
- **An algebraic architecture.** The four layers the algebra implies (storage, surface, RBO, CBO), the logical-to-physical bridge table, and the optimizer pipeline (Section 5.6).

The paper is intentionally foundational. We are not claiming the final word on agentic memory stores; we are claiming that the field will benefit from a shared algebraic vocabulary, just as it has in the relational world.

2 Extended Algebra

2.1 Background

Relational algebra and the classical optimizer. The relational model [1] and its algebra remain the dominant formal substrate for declarative query languages; Codd showed that a small set of operators (σ, π, product, union, difference, rename) is sufficient for any first-order query. The second pillar is **query optimization**: Selinger's cost-based System R optimizer [2] enumerated alternative orderings under a cardinality model; the **Volcano** [3] and **Cascades** [4] frameworks generalized this to a rule-based outer loop driving a cost-based inner search, with the algebra as the API. Two ideas from this tradition are central: an *equational theory* of plan identities is the foundation for principled rewriting, and *logical–physical separation* allows physical innovations to be plugged in without changing semantics.

Vector search and modern IR. Information retrieval contributed the *vector space model* [5] and BM25 [6] as principled relevance scoring functions. The neural era added dense embeddings [7, 8] and approximate nearest neighbour (ANN) indexes for sub-linear search: HNSW graphs [9] are the dominant production choice, with IVF inverted files and brute-force linear scan as alternatives. A crucial property of ANN indexes is that they are *approximate*: the result of an HNSW range query is not the exact set within distance τ but an approximation whose quality depends on the index parameters (`ef`/`M` for HNSW, `nlist`/`nprobe` for IVF). This approximation is invisible at the algebraic level but central to the cost model: a high-recall ANN plan costs more than a low-recall one, and the optimizer must trade recall against latency.

Hybrid query systems. The last decade has produced three lines of *hybrid* systems. **Vector-aware RDBMS extensions** (PostgreSQL `pgvector`, DuckDB `vss`, Oracle AI Vector Search) add a vector type, an ANN index, and distance predicates at the function-call level, with no first-class identities and no cost model for approximation. **Purpose-built hybrid stores** (Pinecone, Weaviate, Qdrant, Milvus, Vespa) are vector-first with metadata filtering bolted on. **Multi-modal query languages** (SQL/MM, SPARQL with full-text, Cypher) mix graph, text, and (in a few cases) vector. The common pattern: every system fuses scores at the application layer rather than extending the algebra that defines what counts as an answer.

The gap. What is missing is an *equational* view of multi-modal memory queries in which vector distance, BM25 relevance, and graph reachability are first-class predicates on a par with $A = c$; in which computed values (distances, scores) are first-class attributes; and in which the boundary between the application's *fusion policy* and the substrate's *evaluation strategy* is explicit and contractually enforced. That is the gap MosaicOpt addresses.

Figure 2 makes the contrast vivid. The left-hand side — multi-path recall — has no plan space, no algebraic identity, no principled rewrite. The right-hand side — the MosaicOpt approach — has all three.

We extend the classical relational algebra with three new kinds of construct: a *materialization* operator that lifts computed values to first-class attributes, first-class multi-modal predicates (vector distance, BM25, graph reachability), and a *graph expansion* operator. Bag semantics are used throughout, matching SQL and avoiding the row-collapse anomalies of set semantics in the presence of duplicate memory units. The logical/physical separation is enforced: physical operators are introduced in Section 5 and never appear in this section.

2.2 Logical/physical separation

Following System R and Cascades, we treat the *logical* algebra as the substrate contract — what the user asks for — and the *physical* algebra as the engine contract — how it is evaluated. Equivalences

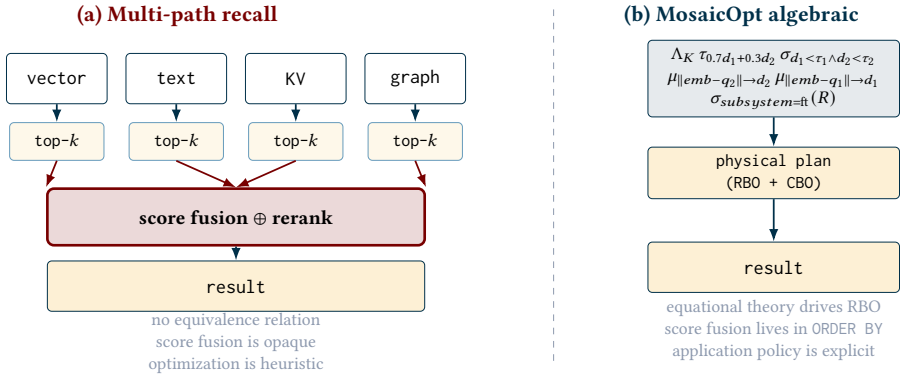


Fig. 2. Multi-path recall (a) versus the MosaicOpt algebraic approach (b). In (a) four independent channels produce ranked candidate lists and application-level score fusion decides the final ranking — there is no equivalence relation over plans, so all optimization is heuristic. In (b) the user writes a single logical plan; the RBO enumerates equivalent rewrites under the equational theory, and the CBO picks a physical realization. The fusion policy is explicit in `ORDER BY`.

Operator	Syntax	Meaning
Base relation	R	The named stored relation.
Selection	$\sigma_\varphi(L)$	Keep rows satisfying φ.
Projection	$\pi_A(L)$	Keep only attributes in A.
Materialization	$\mu_{e\to c}(L)$	Per row, compute e; bind to c.
Join	$L \bowtie_\psi M$	Pair rows with ψ true.
Union / Inter / Diff	$L \cup M$, $L \cap M$, $L - M$	Bag set ops on compat. schemas.
Rename	$\rho_{a\to b}(L)$	Rename attribute a to b.
Ordering	$\tau_O(L)$	Order rows by expression O.
Limit	$\Lambda_K(L)$	First K rows of an ordered input.
Graph expansion	$\Gamma_{g,d}(L)$	From node-id, traverse g to depth d.

Table 1. Logical operators of the extended algebra.

hold in the logical algebra; physical operators are *implementations* of logical operators, and a logical equivalence does not in general imply that two physical realizations are interchangeable (Section 3.4, N1–N3). This separation is what makes the system *composable*: the ANN family (HNSW, IVF, scan) can be added, swapped, or replaced without changing the language the user sees.

2.3 Operator inventory

A *logical plan* is a term over the following operators. R denotes a stored base relation. L, M denote logical plans. φ denotes a predicate, e a materialization expression, A a set of attribute names, O an ordering expression, K a non-negative integer, c a fresh attribute name, g a graph rule (a labelled-edge pattern), and d a non-negative integer depth.

Definition 1 (Bag semantics)**.** A relation is a multiset of tuples. All operators are multiset-preserving: σ, μ, Γ preserve row multiplicity; projection preserves multiplicity; set operations combine multiplicities additively for $\cup$, by min for $\cap$, and by subtraction (clamped at 0) for $-$. τ_O produces a totally ordered sequence; Λ_K truncates it.

Bag semantics was chosen because the substrate must be able to express "the same memory unit appears N times" (e.g. an episodic trace where the same event is recorded with several slightly different timestamps), and because it matches the SQL semantics that application code already expects.

2.4 Predicate forms

A predicate φ is a Boolean combination of atoms. The atom forms are: (i) *classical* ($A = c$, $A < c$, $A > c$, $A \in S$); (ii) *vector* ($\|V_a - q\| < \tau$ where V_a is a vector attribute, q a query-time bound vector, and $\tau \geq 0$ a threshold); (iii) *text* (BM25$(t, q) > \theta$ with parameters (k_1, b) from [6]); (iv) *graph* (reachable(u, v), edge(u, v, ℓ)); and (v) *Boolean combinations* ($\wedge$, $\vee$, $\neg$). *Query-time bound parameters* (q, τ, θ) are first-class values distinct from input attributes, making predicates safely late-binding: the same algebra expression serves all query instances, only the bound parameters change.

2.5 Materialization: motivation, expressions, and semantics

Inlining per-column distances in `ORDER BY` (e.g. `ORDER BY` $\|V_a - q_a\| + \|V_b - q_b\|$) is permitted by SQL extensions, but it forces the application to write the distance expression everywhere it is used and leaves the optimizer no room to share computation between a `WHERE` filter and an `ORDER BY` rank. The materialization operator $\mu_{e \to c}(L)$ binds a per-row expression e to a fresh attribute name c, making the materialized value referenceable by downstream σ, π, τ, and μ (see Section 2.7 for an example). Valid forms of e are classical ($A + B$, $f(A)$), vector ($\|V_a - q\|$), text (BM25(t, q)), and Boolean or arithmetic combinations. The fresh-name condition (Def. 2) requires c to be absent from schema(L); ρ is provided to free names.

Definition 2 (Fresh-name condition)**.** c in $\mu_{e \to c}(L)$ must not already appear in schema(L). ρ is provided to free names if needed.

The semantics is the multiset $\{\, r \oplus \{c \mapsto e(r)\} : r \in L \,\}$. If $e(r)$ is undefined (e.g. distance to a null vector), the row is dropped — consistent with SQL's `NULL` handling and essential for safety in the presence of missing embeddings.

2.6 Graph expansion

$\Gamma_{g,d}(L)$ takes a plan whose rows contain a node-id attribute (u, say) and a graph attribute, applies graph rule g up to depth d, and returns the *target* nodes of those traversals (joined with the originating row). The depth is inclusive of intermediate hops. g is a labelled-edge pattern: a regular expression over edge labels, evaluated against the graph. The graph itself is supplied as a relation-valued attribute; the expansion is a join between L and the traversal result.

Γ is intentionally narrow. We do not attempt to encode path queries as selection predicates; this keeps the operator set small and the rewrite theory tractable (Section 3, G1). Path information can be recovered by a μ that records the path taken.

2.7 Example queries

We use a single running example throughout. Let *Mem* be a memory relation with schema *Mem*(*id*, *text*, *ts*, ***emb***, ***emb_alt***, *user*, *subsystem*) and *KG* a graph relation *KG*(*src*, *dst*, *label*). The query "find similar passages in the fault-tolerance subsystem from the last 30 days, ranked by a

weighted sum of two distances, top 10" is

$$
\begin{aligned}
Q_1 &= \mathit{Mem},\\
Q_2 &= \sigma_{\mathit{subsystem}=\text{'ft'} \wedge \mathit{ts} > \text{now}()-30d}(Q_1),\\
Q_3 &= \mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\| \to d_1}\ \mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\| \to d_2}(Q_2),\\
Q_4 &= \sigma_{d_1<\tau_1 \wedge d_2<\tau_2}(Q_3),\\
Q_5 &= \tau_{0.7\cdot d_1+0.3\cdot d_2}(Q_4),\\
Q_6 &= \Lambda_{10}(Q_5).
\end{aligned}
$$

The algebra's job is to make this expression *rewriteable* — to compute d_1, d_2 only on rows that survive the filter (M1) and to enable top-K early termination (T1). A graph-aware variant intersects the result with a two-hop Γ expansion $R_2 = \Gamma_{\mathit{label}=\text{'derived_from'},\, d=2}(R_1)$.

3 Equational Theory

The optimizer (Section 4.7) is driven by an *equational theory*: a set of identities over logical plans. The same theory also records, explicitly, the identities that classical intuition suggests but approximation breaks — the *non-identities* — so the optimizer knows not to apply them. This section unifies both sides. We first give an overview (4.1), then the inherited classical axioms (4.2), then the new identities that the extended operators license (4.3), then the derived rewrites the optimizer uses (4.4), and finally the non-identities with counterexamples (4.5).

3.1 Overview

The equational theory plays two roles. *First*, it is the foundation for the rule-based optimizer (Section 4.7): every rewrite is justified by an identity (M1–M3, V1, J1, T1, G1 or a derived combination), so the optimizer's behaviour is auditable. *Second*, the *non*-identities (N1–N3) are the discipline that prevents the optimizer from applying rewrites that hold in classical relational algebra but break under approximation, asymmetric physical costs, or distributed semantics. The cost model is responsible for the physical cost of every rewrite; the equational theory is responsible for its *correctness*. Both identities and non-identities are first-class data: the RBO loads them, the cost model consults them, and the executor relies on them for safety of adaptive re-planning.

3.2 Foundational axioms

The seven identities below are licensed by the extended algebra; each is stated with its applicability conditions, and one-sentence proof pointers (full proofs in Appendix A.1). The classical identities A1–A6 (selection composition, selection over union, projection idempotence, selection–join pushdown, projection–join pushdown, rename involution) are assumed without re-proof — they hold for the extended predicate form.

Theorem 1 (M1 — Selection commutes with materialization on the materialized column)**.** *Where c is a fresh attribute name and e is an expression depending only on attributes of L:*

$$\sigma_{c<\tau}\, \mu_{e\to c}(L) \;=\; \mu_{e\to c}\, \sigma_{e<\tau}(L).$$

(*Proof in Appendix A.1.*)

The M1 saving is dramatic for tight vector ranges: for selectivity 10^{-3} or less, e is computed on $\sim 10^3$ fewer rows.

Theorem 2 (M2 — Materialization is commutative)**.** *For fresh names c_1, c_2 and expressions e_1, e_2 depending only on L's attributes (and not on each other's outputs):*

$$\mu_{e_1\to c_1}\, \mu_{e_2\to c_2}(L) \;=\; \mu_{e_2\to c_2}\, \mu_{e_1\to c_1}(L).$$

(*Proof in Appendix A.1.*)

Theorem 3 (M3 — Materialization through projection)**.** *For fresh c and expression e depending only on attributes in A:*

$$\mu_{e \to c}\, \pi_A(L) \;=\; \pi_{A \cup \{c\}}\, \mu_{e \to c}(L).$$

(*Proof in Appendix A.1.*)

Theorem 4 (V1 — Vector selections on distinct columns commute)**.** *For distinct vector-valued attributes V_a, V_b and corresponding bound parameters and thresholds:*

$$\sigma_{\|V_a - q_a\| < \tau_a}\, \sigma_{\|V_b - q_b\| < \tau_b}(L) \;=\; \sigma_{\|V_b - q_b\| < \tau_b}\, \sigma_{\|V_a - q_a\| < \tau_a}(L).$$

(*Proof in Appendix A.1.*)

Remark 1. V1 is a *correctness* identity: the two plans produce the same logical result, but not the same cost — the order of vector selections affects ANN probe counts and intermediate result sizes. The optimizer must keep both orderings and let the cost model choose.

Theorem 5 (J1 — Vector selection pushdown through join)**.** *Where V_a is an attribute of L and ψ is the join predicate:*

$$\sigma_{\|V_a - q\| < \tau}\, (L \bowtie_\psi M) \;=\; (\sigma_{\|V_a - q\| < \tau}(L)) \bowtie_\psi M.$$

(*Proof in Appendix A.1.*)

Theorem 6 (T1 — Limit after order equals top-K)**.** *Where O is a total ordering expression:*

$$\tau_O\, \Lambda_K(L) \;=\; \mathrm{topK}_O(L, K).$$

(*Proof in Appendix A.1.*)

Theorem 7 (G1 — Selection pushdown through graph expansion)**.** *Where φ references only attributes of L (and not attributes of the graph rule's traversal result):*

$$\sigma_\varphi\, \Gamma_{g,d}(L) \;=\; \Gamma_{g,d}\, \sigma_\varphi(L).$$

(*Proof in Appendix A.1.*)

3.3 Derived identities

By composing the identities, additional rewrites become available. We name two that the optimizer uses directly:

- **J1+M1 (vector selection before materialization).** Combining J1 and M1 allows a join-side vector selection to be pushed below a materialization that binds a distance for sorting:

$$\begin{aligned} \sigma_{\|V-q\|<\tau}\, \mu_{\|V-q\| \to d}\, (L \bowtie M) &= \mu_{\|V-q\| \to d}\, \sigma_{\|V-q\|<\tau}\, (L \bowtie M) \quad \text{(M1)} \\ &= \mu_{\|V-q\| \to d}\, \left(\sigma_{\|V-q\|<\tau}(L) \bowtie M\right) \quad \text{(J1)} \end{aligned}$$

  The plan now computes the distance only on the surviving L rows.
- **T1 (topK replaces $\tau + \Lambda$).** Whenever the algebra has $\tau_O\, \Lambda_K$, the CBO can replace the pair with a single `topK` physical operator (Section 5).

The algebra is closed under composition: the output of any operator is a logical plan usable as input to any other. The schema rules ensure that the attributes a downstream operator references are either attributes of the input or have been introduced by a μ in the pipeline. The rewrite rules documented in Sections 3.2–3.3 — together with the standard axioms A1–A6 — therefore suffice to cover the optimization of all plans whose operators draw from the inventory of Section 2.

3.4 Non-identities and counterexamples

Some rules that look natural in a relational setting **do not hold** in the extended algebra. We document these explicitly so the optimizer knows not to apply them. The same logical plan in two realizations can have different outputs; the optimizer must treat this as a *cost* phenomenon, not a *correctness* phenomenon.

3.4.1 *N1: Approximation breaks logical equivalence.*

Theorem 8 (N1 — ANN approximation breaks logical equivalence)**.** *Logical $\sigma_{\|V-q\|<\tau}(R)$ is exactly defined; physical $H_{V,q,\tau}(R)$ under HNSW or IVF is approximate (misses true neighbours, may include non-neighbours). A logical equivalence does not imply that two physical realizations preserve it.*

Counterexample 1. A row with $\|\boldsymbol{v}-\boldsymbol{q}\| = 0.99\tau$ may be missed by a small-`ef` HNSW but returned by an exact linear scan: two valid implementations of the same logical selection disagree. The optimizer must model approximation as a first-class *cost dimension* with quantified recall/precision.

3.4.2 *N2: Similarity join asymmetry.*

Theorem 9 (N2 — Similarity join asymmetry)**.** *$R \bowtie_{\|V_a-V_b\|<\tau} S$ and $S \bowtie_{\|V_b-V_a\|<\tau} R$ are logically equivalent; the two physical sides are not interchangeable in cost.*

Counterexample 2. Probing a 10^6-row HNSW on R with each row of a 10^3-row S costs 10^3 probes; the reverse (probe S for each row of R) costs 10^6 — three orders of magnitude apart. The RBO generates both orderings; only the CBO can decide which is cheaper.

3.4.3 *N3: Limit and order do not distribute over union.*

Theorem 10 (N3 — Limit and order do not distribute over union)**.** $\Lambda_K\, \tau_O\, (R \cup S) \;\neq\; \Lambda_K\, \tau_O(R) \cup \Lambda_K\, \tau_O(S)$ *in general.*

Counterexample 3. With R at score 0 and S at score 10, the LHS top-5 are the 5 rows of R; the RHS gives 5 from R *plus* 5 from S — a 10-row result. The identity would be useful for distributed topK, but does not hold.

3.5 Convergent rewriting

The equational theory of Sections 4.2–4.5 can be oriented into a **rewriting system** and analyzed for termination and confluence. Each identity is oriented toward the optimized form, giving an *abstract rewriting system* (ARS) $\to_{\text{RBO}}$ on logical plans; the optimizer of Section 4.7 is an interpreter for $\to_{\text{RBO}}$. Define the weight $w(P) = \alpha s(P) + \beta r(P) + \gamma p(P)$ where s counts σs on μ-bound attributes (M1-fixable), r counts redundant μs, and p counts πs that drop needed attributes (M3-fixable). Every oriented rule strictly decreases w, and $w(P) \leq (\alpha+\beta+\gamma)\cdot|P|$, so the ARS terminates. The critical pairs from overlapping redexes (M1∩M1, M1∩J1, G1∩A4, M2∩V1, T1∩M1) all join by further rewrites, giving local confluence. By Newman's lemma [12], termination + local confluence imply confluence and a unique normal form:

Theorem 11 (RBO Confluence)**.** *For any starting logical plan P and any two RBO rewrite sequences $P \to^*_{\text{RBO}} P^*$ and $P \to^*_{\text{RBO}} P^{**}$, there exists a plan P^∞ with $P^* \to^*_{\text{RBO}} P^\infty$ and $P^{**} \to^*_{\text{RBO}} P^\infty$. The normal form is unique: it depends only on P, not on rewrite order.*

(*Proof in Appendix A.2.*)

Theorem 11 is the formal correctness guarantee of the RBO — the optimizer can be implemented as a simple greedy loop (apply any rule that decreases w) without losing correctness. The CBO inherits confluence "for free": its search is over the unique canonical form, so its choice of physical realization is well-defined and reproducible.

4 SQL Surface and Rule-Based Optimization

The algebra of Section 2 is the optimizer's working language; application code writes SQL, in a small extension that is the locked-in query interface for the prototype. This section makes the two surfaces explicit, gives the desugaring from SQL into the algebra, and states the RBO that drives the optimizer.

4.1 Motivation: two equivalent surfaces

Application code, by tooling and decades of investment, is expressed in SQL. The architecture is a *thin SQL surface* that desugars into the algebra, with a *round-trip* property: every SQL query expresses an algebra plan, and every algebra plan has a SQL rendering. The optimizer manipulates the algebra; the parser is the only component that sees SQL.

4.2 DDL: vector type and indexes

The SQL surface adds one *type* and three *index* DDL clauses. Vectors are first-class column types (no separate vector store); indexes are declared on the same relation that holds the data.

```
-- Vector column with optional explicit metric
ALTER TABLE memory
    ADD COLUMN emb VECTOR(768);
ALTER TABLE memory
    ADD COLUMN emb_alt VECTOR(768, METRIC := 'cosine');

-- ANN index on a vector column (HNSW, IVF, or scan-fallback)
CREATE INDEX idx_emb_cos
    ON memory USING hnsw (emb vector_cosine_ops);
CREATE INDEX idx_emb_ivf
    ON memory USING ivf (emb vector_cosine_ops);

-- BM25 inverted index on a text column
CREATE INDEX idx_text_bm25
    ON memory USING bm25 (text);
```

The `VECTOR(N)` type is a typed array of N 32-bit floats; the optional `METRIC` clause names the distance metric and the CBO prefers the matching ANN index. The `USING hnsw`, `USING ivf`, and `USING bm25` index methods are the physical access paths of Section 5 exposed as DDL.

4.3 WHERE: multi-modal predicates

The `WHERE` clause of a query is the primary surface for multi-modal predicates. We adopt **SQL/MM-style syntax** for all three new predicate kinds (vector, text, graph) and *keep* pgvector's distance operators for low-level use:

```
-- Vector range predicate (SQL/MM-style, first-class)
WHERE emb IS NEAR :q1 WITHIN DISTANCE :tau1
WHERE text MATCHES :q1 WITH SCORE > :theta
WHERE id IS REACHABLE FROM :n WITHIN 2 HOPS VIA 'derived_from'
```

The SQL/MM form is the user's explicit preference: function-style (`distance(emb, :q) < :tau`) reads as a *user-defined function* the engine does not understand, while `IS NEAR ... WITHIN DISTANCE` reads as a *first-class* construct the engine is committed to understanding. The same logic

applies to `MATCHES ... WITH SCORE` and `IS REACHABLE ... VIA`. We retain pgvector's `<=>`, `<->`, `<#>` operators for backward compatibility and as the natural surface for `ORDER BY` expressions that need the raw distance.

4.4 Materialization via existing AS

The materialization operator μ (Section 2) binds a computed value as a first-class attribute. The SQL surface does *not* add a new `LET` clause; the existing `AS` alias is the materialization syntax. The optimizer recognizes when an `AS`-bound value is reused in `WHERE` or `ORDER BY` and triggers the M1 rewrite (Section 3.2) automatically:

```
SELECT
    id, text, ts, subsystem,
    emb <=> :q1     AS d1,    -- materializes distance to :q1
    emb_alt <=> :q2 AS d2     -- materializes distance to :q2
FROM memory;
```

When the plan is later used in a `WHERE d1 < :tau1` predicate or an `ORDER BY 0.7 * d1 + 0.3 * d2` expression, the optimizer pushes the σ below the μ (M1) — unless the user opted into eager materialization via the `MATERIALIZED` hint (a thin override; the default is for the optimizer to decide, with the equational theory of Section 3 as the formal contract).

4.5 Graph expansion via recursive CTE

Graph traversal is exposed as a standard SQL `WITH RECURSIVE` clause — no new syntax. The optimizer handles the recursion and feeds the resulting fixed point into the rest of the plan as a normal relation:

```
WITH RECURSIVE reachable AS (
    SELECT :n AS id, 0 AS depth
    UNION
    SELECT edge.dst, reachable.depth + 1
    FROM edge JOIN reachable ON edge.src = reachable.id
    WHERE reachable.depth < 2
)
SELECT m.*
FROM memory m
JOIN reachable r ON m.id = r.id
WHERE m.subsystem = 'fault-tolerance';
```

The recursive CTE is the standard SQL construct for transitive closure; the optimizer recognizes the `edge.src = reachable.id` join as a Γ-style expansion and emits the corresponding Γ operator (Section 2), with `WHERE reachable.depth < 2` as the depth bound. We do *not* introduce a new `TRAVERSE` keyword; the recursive CTE is a familiar idiom.

4.6 Desugaring rules: SQL → algebra

The desugaring is a recursive function $\mathcal{D}$ (read "desugar of") that maps SQL fragments to algebra terms; we use $\mathcal{D}\langle\cdot\rangle$ for SQL-source and $\mathcal{D}\langle\cdot\rangle_{\text{pred}}$ for predicates. The function is total on the surface syntax of Sections 4.2–4.5 and is the *only* place where the parser and the algebra meet. We state the rules informally; the implementation is the recursive-descent parser.

D1. Projection + filter + order + limit. For

```
SELECT expr AS c, ...
FROM L
WHERE phi
ORDER BY O
LIMIT K
```

the desugaring is

$$\mathcal{D}\langle\cdot\rangle \;=\; \Lambda_K\big(\tau_O\big(\sigma_\varphi\big(\mu_{e\to c}\big(\mathcal{D}\langle L\rangle\big)\big)\big)\big),$$

with μ chaining for multiple aliased expressions; `ORDER BY` accepts both simple column references and complex expressions over materialized columns.

D2. Vector predicate. $\mathcal{D}\langle$ emb IS NEAR :q WITHIN DISTANCE :t $\rangle_{\text{pred}} = \sigma_{\|emb-q\|<t}$. The bound `:q` and threshold `:t` are query-time parameters; the desugared form is a first-class algebra expression on which J1 and M1 apply.

D3. Text predicate. $\mathcal{D}\langle$ text MATCHES :q WITH SCORE > :$\theta\rangle_{\text{pred}} = \sigma_{\text{BM25}(text,q)>\theta}$. The (k_1, b) parameters are carried with the BM25 index; the CBO's inverted-index bridge applies directly to the desugared form.

D4. Graph predicate (and recursive CTE). $\mathcal{D}\langle$ id IS REACHABLE FROM :n WITHIN k HOPS VIA 'l' $\rangle_{\text{pred}}$ $\sigma_{id\,\in\,\pi_{dst}(\Gamma_{label='l',d=k}(\sigma_{id=:n}(Mem)))}$; a `WITH RECURSIVE` clause is first expanded to its least fixed point and the result is translated by D1–D3. The optimizer recognizes the `lfp` form and emits a Γ operator with the appropriate depth and label.

Each rule preserves semantics *trivially*, by construction: each is the definition of the corresponding SQL/MM clause in terms of the algebra's operator semantics.

4.7 From algebra to RBO: role, transformations, plan space

The rule-based optimizer (RBO) is the first phase of the optimizer pipeline. It operates on **logical plans** and applies the identities of Section 3 to enumerate equivalent plans. The output is a (typically modest) set of logically equivalent plans that the CBO will then consider for realization.

4.8 RBO role

The RBO's job is **not** to pick a plan. Its job is to generate a *plan space* in which the CBO can search effectively. The RBO uses the equational theory to: (i) apply selection pushdown (A4, J1, G1); (ii) commute vector and classical selections (V1, A1) so the more selective predicate runs first; (iii) reorder materializations (M2) so independent materializations can be evaluated in any order; (iv) expose topK opportunities (T1) by collapsing $\tau_O\,\Lambda_K$ into a single `topK` operator that the physical planner can implement with a bounded heap.

The RBO does *not* decide which access path (HNSW/IVF/scan for vector, B-tree vs. heap for structured) to use. It only reshapes the logical plan.

4.9 Example transformations

Returning to the Q_1–Q_6 example of Section 2, the input logical plan is

$$\Lambda_{10}\;\tau_{0.7\cdot d_1+0.3\cdot d_2}\;\sigma_{d_1<\tau_1\wedge d_2<\tau_2}$$
$$\mu_{\|\boldsymbol{emb_alt}-\boldsymbol{q}_2\|\to d_2}\;\mu_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|\to d_1}\;\sigma_{subsystem=\text{`ft'}\wedge ts>\text{now}()-30d}(Mem).$$

The RBO applies the following rewrites in order:

(1) **Selection–materialization commutativity** (M1), per predicate: each $\sigma_{d_1<\tau_1}$ is rewritten to $\mu_{\ldots\to d_1}\,\sigma_{\|\boldsymbol{emb}-\boldsymbol{q}_1\|<\tau_1}$. Each μ is moved above only the σ that consumes its output, so the distance is computed on the surviving rows.

(2) **Vector–vector commutation** (V1): the two $\sigma_{\|\cdot\|<\tau}$ predicates commute, so the RBO records both orderings and the CBO picks the one with the cheaper first probe.

The full trace also fires A1, A4, T1, and M2 (selection composition, classical predicate pushdown, topK fusion, and materialization reordering), but M1 and V1 are the rewrites that exploit the new operators. The output is a small set of logically equivalent plans; the CBO chooses among them.

4.10 Plan space pruning

Without the RBO, the CBO would consider every shape of every plan: for n joins, $O(n!)$; for k materializations, $O(k!)$ per join ordering. The RBO prunes this space in two ways: it groups plans into *equivalence classes* under the rewriting rules (the CBO need not consider plans the RBO has marked suboptimal) and chooses a *canonical form* for each class (selections pushed down, materializations clustered, topK fused). The CBO's search is then over plan shapes and physical realizations jointly, but the number of shapes is small — typically 1–3 for the agentic-memory queries we have explored.

4.11 Plan space and RBO complexity

For a query Q, the *plan space* $\mathrm{Space}(Q)$ is the set of logical plans reachable from Q by $\to_{\mathrm{RBO}}$. By Theorem 11 the plan space is finite and its extremal elements are the normal form of Q (unique up to the V1/G1 commutativity classes). For n operator occurrences, $|\mathrm{Space}(Q)| \le c^n$ for a constant c depending on the rule set: the exponent comes from the M2, V1, G1, and A4 commutation classes. In the worst case $c = O(n!)$; for typical agentic-memory queries the plan space is $O(10^3)$ to $O(10^5)$.

The RBO's effective runtime is $O(|Q|)$: the number of rule applications is bounded by $w(Q)$ (every application strictly decreases w), and $w(Q) \le (\alpha + \beta + \gamma) \cdot |Q|$. Two pruning heuristics — **rule preconditioning** (each rule carries a precondition that gates its firing) and **memoization** (a Cascades-style memo caches explored subplans) — reduce the constant factor with no effect on correctness.

5 Cost Model, CBO, and Architecture

The cost-based optimizer (CBO) is the second phase. It takes the equivalence classes produced by the RBO and selects a *physical plan* — a tree of physical operators — under a cost model.

5.1 Physical operator inventory

The physical operators are the *implementations* of the logical operators. Each physical operator carries parameters that the CBO can tune.

5.2 Cost dimensions

The cost model has four dimensions. The first three are classical; the fourth is new.

- **I/O cost.** Page reads and writes; dominant for cold data, mitigated by indexes.
- **CPU cost.** Distance computations, BM25 scoring, materialization; for vector queries, per-probe distance is the dominant component.
- **Memory cost.** Working set size (intermediate hash tables, priority queue for `TopK`).
- **Approximation cost.** Quality penalty of an approximate physical operator: HNSW: $1-\mathrm{recall}(ef, M)$; IVF: $1 - \mathrm{recall}(nprobe, nlist)$; scan: recall $= 1$. The CBO treats approximation as a constraint ("recall $\ge \rho_{\min}$") or as a weighted cost term.

The cost is additive across operators in a pipeline, weighted by estimated cardinalities at each boundary.

Physical	Implements	Notes
$H_{V,q,\tau}$	$\sigma_{\|V-q\|<\tau}$	ANN range scan; strategy ∈ {HNSW, IVF, scan}; HNSW params `ef`, M; IVF params `nlist`, `nprobe`; scan cost linear in $\|R\|$.
$B_{col,r}$	$\sigma_{col \in r}$	B-tree range scan.
$F(R)$	base scan	Sequential file scan.
$I_{t,q,\theta}$	$\sigma_{\text{BM25}>\theta}$	Inverted index probe.
$G_{g,d,dir}$	$\Gamma_{g,d}$	Graph traversal; *dir* is direction.
Sort(O)	τ_O	Full sort.
TopK(O,K)	$\tau_O \Lambda_K$ (T1)	Bounded-heap top-K.
Materialize$(e \to c)$	$\mu_{e\to c}$	Computes e and binds c.
HashJoin, NLJ, MergeJoin	$\bowtie_\psi$	Standard join algorithms.
Partition, HashBuild, HashProbe	hash-based operators	Standard enforcers.

Table 2. Physical operators and the logical operators they implement.

5.3 Logical-to-physical bridge

For each logical operator, the CBO enumerates candidate physical realizations in the Cascades sense: $\sigma_{\|V-q\|<\tau}(R) \to H_{V,q,\tau}(R)$ (HNSW/IVF/scan, or HNSW-then-exact-refine); $\sigma_{\text{BM25}>\theta}(R) \to I_{t,q,\theta}(R)$ if an inverted index exists, else full scan with scoring; $\tau_O \Lambda_K(R) \to \text{TopK}(O,K)(R)$ per T1, or Sort + Limit when sort cost is amortized.

5.4 Adaptive execution

Static cost estimates are notoriously wrong for vector workloads (ANN recall varies with the query, BM25 selectivity is hard to predict from small samples). The prototype supports **adaptive execution**: at well-defined plan points (post-HNSW-candidate, post-inverted-probe), the executor re-consults the cost model with observed cardinalities and may switch to a different sub-plan. The algebra is unchanged; only the physical subtree is replaced. Adaptive switches are *safe by construction* because the two subtrees are both implementations of the same logical operator.

5.5 Formal cost model and CBO complexity

The cost model of Section 5 is a *vector* of four components that can be reasoned about symbolically.

Cost as a 4-tuple. For a logical plan P, define

$$\text{Cost}(P) \;=\; (\text{io}(P),\ \text{cpu}(P),\ \text{mem}(P),\ \text{approx}(P)) \;\in\; \mathbb{R}^4_{\geq 0}.$$

The CBO's objective is to choose, among all physical realizations of the canonical form of P, the one that minimizes a weighted sum of the four components (weights set by application policy), subject to a constraint $\text{approx}(P) \leq \varepsilon$ for an application-chosen $\varepsilon \in [0,1]$. Equivalently, the CBO minimizes a Pareto front: a plan P_1 Pareto-dominates P_2 if $\text{Cost}(P_1) \leq \text{Cost}(P_2)$ componentwise and at least one component is strictly smaller.

Per-operator cost expressions. Each physical operator has a symbolic cost expression; the cost of a plan is the sum of its operators' costs, adjusted by the input cardinality at each operator's boundary. The full cost-vector table is in Appendix A.4.

Total cost. For a plan tree $T = op(T_1, \ldots, T_k)$, $\text{Cost}(T) = \text{Cost}_{op}(T) + \sum_i \text{Cost}(T_i) + \text{io}_{\text{materialize}}(T)$: additive over operator subtrees, with a spill term for forced materializations.

CBO complexity. $|\text{Space}_{\text{CBO}}(Q)| \leq |\text{Space}_{\text{RBO}}(Q)| \cdot |\text{Phys}|^n$ where n is the operator count. The CBO applies three standard pruning techniques: (i) **Selinger-style DP** for the $\bowtie$-fragment ($O(n^2)$ for left-deep trees); (ii) **greedy selectivity ordering** for vector predicates (skip the $k!$ permutation exploration when selectivities are far apart); (iii) **top-K pruning** when τ is followed by Λ_K. The polynomial-subclass boundary is open (Section 6.3).

5.6 Architectural implications: layers and pipeline

A system that instantiates the MosaicOpt algebra must provide four layers:

(1) **Storage layer.** Physical realizations of the column types: vector columns as one of three ANN strategies (HNSW graph, IVF inverted file, brute-force scan), text columns as BM25 inverted indexes, structured columns as B-trees, graph relations as CSR or adjacency-map structures. Each storage primitive is a *physical operator* in Section 5.
(2) **Surface layer.** The desugaring rules of Section 4.6 compile SQL — with the `IS NEAR`, `MATCHES`, and `IS REACHABLE` extensions — into logical algebra plans. The surface layer is the *only* component that sees SQL.
(3) **Rule-based optimizer (RBO) layer.** Applies the equational rules of Section 3 to rewrite logical plans into their normal form (Theorem 11).
(4) **Cost-based optimizer (CBO) layer.** Enumerates physical implementations of the logical normal form and selects the minimum-cost realization under the 4-tuple cost model of Section 5.5.

The four layers are *strictly ordered* (each consumes only the output of the one above), making the system *compositional*: a new storage primitive is a new bridge-table entry; a new SQL clause is a new desugaring rule; a new optimizer rule is a new equational identity.

5.7 Storage and the logical-to-physical bridge

The bridge between the logical algebra (what the optimizer rewrites) and the physical algebra (what the executor runs) is a *table of logical-to-physical mappings* indexed by predicate form — the single point of contact between the abstract algebra and the concrete storage layer.

Logical operator	Realization (storage primitive)
$\sigma_{\|V-q\|<\tau}(R)$	$H_{V,q,\tau}(R)$ (HNSW/IVF/scan)
$\sigma_{\|V-q\|<\tau}(R)$ (no index)	$\mu_{\|V-q\|\to d}\, F(R);\ \sigma_{d<\tau}$ (linear scan)
$\sigma_{\text{BM25}(t,q)>\theta}(R)$	$I_{t,q,\theta}(R)$ (inverted index)
$\sigma_{A\in range}(R)$	$B_{A,range}(R)$ (B-tree)
$\Gamma_{g,d}(L)$	$G_{g,d,dir}(L)$ (graph traversal)
$\tau_O\, \Lambda_K(L)$	$\text{TopK}(O,K)(L)$ (bounded heap)
$\tau_O(L)$ (no Λ)	$\text{Sort}(O)(L)$ (full sort)
$\mu_{e\to c}(L)$	$\text{Materialize}(e \to c)(L)$
$L \bowtie_{\psi} M$	HashJoin / NLJ / MergeJoin
$F(R)$	$F(R)$ (sequential scan)

Table 3. Logical-to-physical mapping table. The CBO's search is over the entries of this table, indexed by the operator's predicate form.

Three observations about this table:

- **Small.** Each logical operator has 1–4 physical realizations (HNSW/IVF/scan, TopK vs. Sort+Limit, join algorithms), keeping the CBO's search space controlled.
- **The only extension point.** A learned index [10] or a more sophisticated ANN variant (product-quantization, HNSW+, scan with pruning) is a new table entry plus a cost-vector entry; no optimizer change is needed.

- **Auditable.** The table is a first-class data structure — not code — so it can be inspected, version-controlled, and tested independently of the optimizer.

5.8 The optimizer pipeline

The optimizer is a three-stage pipeline: (1) **SQL → logical algebra** via the desugaring rules (Section 4.6); (2) **logical → logical (RBO)** via the equational rules, terminating and confluent (Theorem 11), producing a unique normal form; (3) **logical → physical (CBO)** via the cost-based search of Section 5.5. Stage 3 is the only one that involves search.

6 Discussion

6.1 Related work

We identify the lineage of MosaicOpt precisely — the goal is not to enumerate all related work but to make clear what is novel.

Classical relational algebra. MosaicOpt inherits from Codd's relational algebra [1] (the small operator set that expresses any first-order query, refined over time and crystallized in SQL) and from the Selinger/Volcano/Cascades line [2–4] (cost-based optimization, extensible rule-based rewriters, the maxim that the algebra is the API). These two pillars are unchanged; what MosaicOpt adds is (i) a new set of identities (M1–M3, V1, J1, T1, G1) for the extended operators, and (ii) an explicit convergent-rewriting analysis (Section 3.5).

Algebraic extensions of the relational model. MosaicOpt continues a long lineage of "extend the algebra, prove identities" work: **nested** relational algebra [19] adds set-valued attributes (MosaicOpt adds first-class materialization); **object-relational** extensions [20] add user-defined types; **temporal** algebras [21, 22] add time-aware predicates; **semi-structured** algebras [23] add tree-structured data and bisimulation-style identities. MosaicOpt extends in a *new direction* — multi-modal (text + vector + graph) in the context of agentic memory — and provides, in one place, a SQL surface and a proven equational theory the prior extensions did not.

Equational theories in query optimization. The classical identities (selection pushdown, join commutativity, projection pushdown) have been in the optimizer's arsenal since System R. The convergent-rewriting analysis of Section 3.5 draws on Newman's lemma [12], Huet's critical-pair analysis [13], and the Terese textbook [14]. The closest database prior is Atzeni and Torlone's [24] algebraic treatment of view maintenance, whose pattern is identical. MosaicOpt's analysis is, to our knowledge, the first such analysis for an algebra that includes vector, text, and graph operators.

6.2 Hybrid query algebra (recent)

The last decade has produced a sustained effort to bring vector and relational together, but with very few *theoretical* contributions. The systems are numerous; the algebras are few: **vector-aware RDBMS extensions** (PostgreSQL `pgvector`, DuckDB `vss`, Oracle AI Vector Search) add a vector type, an ANN index, and distance predicates at the function-call level — systems, not theories; **purpose-built hybrid stores** (Pinecone, Weaviate, Qdrant, Milvus, Vespa) are vector-first with metadata filtering bolted on and expose no named identities; **multi-modal query languages** (SQL/MM, SPARQL with full-text, Cypher) mix graph, text, and (in a few cases) vector, but SPARQL's vector fusion remains open. The common pattern: every system fuses scores at the application layer. MosaicOpt's contribution is to make the fusion *first-class*: a materialization expression is a syntactic construct, the optimizer sees it, the cost model accounts for it, and the equational theory of Section 3 applies.

6.3 Open theoretical problems

Each layer of the algebra, equational theory, and optimizer opens new theoretical problems that the algebra makes accessible but does not yet solve. We enumerate the most directly consequential.

Complexity of CBO.. Classical join ordering is solvable in $O(n^2)$ for left-deep trees [2] but NP-hard for bushy trees [17]. The CBO of Section 5.5 adds three new dimensions: (i) *vector selections* as first-class operators contribute a *continuous* (`ef`-, `nprobe`-parameterized) choice, not a discrete one; (ii) *graph expansion* has a direction-parameterized realization whose cost depends on *graph density* at the expansion point; (iii) *materialization sharing* is a partition problem — each μ may be shared with another μ or kept distinct, and the CBO must decide. We conjecture NP-hardness in the general case and conjecture polynomial-time subclasses for queries that bound the number of graph expansions, the depth of materialization sharing, and the cardinality of physical realizations per operator. Identifying these subclasses — and proving the tractable/intractable boundary — is a central open problem.

Adaptive optimization. The cost model is static — evaluated once at planning time — but wrong in practice (cardinality drifts, ANN recall varies with the query, BM25 selectivity is hard to predict). *Adaptive optimization* [3] must interact correctly with the algebra's convergent rewriting: the re-planned logical plan must be a "refinement" of the original, and any non-normal-form subplan the CBO switches to must later re-rewrite — a property we conjecture holds locally to the switched subtree.

Distributed execution. A distributed extension requires a network-cost model, partition-aware physical operators (e.g. a partition-aware TopK that respects N3), and an extension of the equational theory to reason about partitioning. The asymmetry is that graph expansion Γ is harder to parallelize than vector selection — a graph expansion must traverse edges across partition boundaries, while a vector selection is partitioned by the index itself. The cost of $\Gamma_{g,d}$ is a function of the partition structure, not just the graph size.

Write paths and memory evolution. The algebra is read-only. A write algebra must model (i) *incremental materialization updates* (when a base row changes, every μ that depends on it must be invalidated or recomputed; the equational theory gives the semantics, incremental maintenance is harder); (ii) *index updates* (ANN and inverted indexes must stay consistent with the base data); and (iii) *cost model drift* (cardinality and selectivity parameters drift and need re-calibration). Writes are the largest open problem in the paper.

6.4 Limitations and out-of-scope

- **No new physical data structures.** ANN indexes, B-trees, and inverted indexes are external; the contribution is the *algebraic* use of these primitives, not their internal design. A learned-index primitive [10] is a drop-in replacement in the bridge table.
- **No new learning-based optimization.** The MosaicOpt contribution is at the *logical* layer; a learned optimizer can *instantiate* the cost model without touching the algebra.
- **No new transaction model.** The algebra is read-only and single-node; transactional and distributed extensions are out of scope and addressed in the open problems above.

These are *scope choices* that keep the contribution focused on the algebraic foundation.

References

[1] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.

[2] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pages 23–34, 1979.
[3] G. Graefe. Volcano — an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994.
[4] G. Graefe. The Cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, 1995.
[5] G. Salton, A. Wong, and C. S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.
[6] S. Robertson, S. Walker, and S. Jones. Okapi at TREC-3. In *NIST Special Publication 500-225: Proceedings of the Third Text REtrieval Conference (TREC-3)*, 1995.
[7] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. In *ICLR Workshop*, 2013.
[8] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP*, 2019.
[9] Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.
[10] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis. The case for learned index structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*, pages 489–504, 2018.
[11] N. Dalvi and D. Suciu. Efficient query evaluation on probabilistic databases. *VLDB Journal*, 16(4):523–544, 2007.
[12] M. H. A. Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942.
[13] G. Huet. Confluent reductions: Abstract properties and applications to term rewriting systems. *Journal of the ACM*, 27(4):797–821, 1980.
[14] Terese. *Term Rewriting Systems.* Cambridge University Press, 2003.
[15] C. J. Date. *An Introduction to Database Systems.* Addison-Wesley, 8th edition, 2003.
[16] J. D. Ullman. *Principles of Database and Knowledge-Base Systems, Volume I.* Computer Science Press, 1988.
[17] T. Ibaraki and T. Kameda. On the optimal nesting order for computing n-relational joins. *ACM Transactions on Database Systems*, 9(3):482–502, 1984.
[18] A. Pellenkoft, C. Galindo-Legaria, and M. Kersten. The complexity of transformation-based join enumeration. In *Proceedings of VLDB*, pages 306–315, 1996.
[19] S. J. Thomas and P. C. Fischer. Nested relational structures. *Advances in Computing Research*, 3:269–307, 1986.
[20] D. Beech. A foundation for evolution from relational to object databases. In *Proceedings of EDBT*, pages 251–270, 1989.
[21] A. Tuzhilin and J. Clifford. A temporal relational algebra as a basis for temporal relational completeness. In *Proceedings of VLDB*, pages 13–23, 1990.
[22] R. T. Snodgrass. *The TSQL2 Temporal Query Language.* Kluwer, 1995.
[23] M. Fernandez, J. Simeon, and P. Wadler. A semi-monad for semi-structured data. In *Proceedings of ICDT*, pages 263–300, 2000.
[24] P. Atzeni and V. Torlone. Management of multiple models in an extensible database design tool. In *Proceedings of EDBT*, pages 79–95, 1996.

A Proofs of Foundational Results

This appendix collects the full proof sketches of the *foundational axioms* (M1–M3, V1, J1, T1, G1) and the two *convergence/complexity* results (RBO Confluence and CBO complexity bound) that are stated in the body without their proofs for compactness.

A.1 Proofs of foundational axioms

M1 — Selection commutes with materialization. Fix $r \in L$. The LHS computes $e(r)$, attaches it as c, and retains $r \oplus \{c \mapsto e(r)\}$ iff $e(r) < \tau$. The RHS evaluates $e(r) < \tau$ directly: if false, r is dropped; if true, the row is extended to $r \oplus \{c \mapsto e(r)\}$ by the μ. The two multisets are equal row-by-row. The freshness of c ensures that the predicate $c < \tau$ in LHS is unambiguous — c is defined by the μ and is not shadowed by any pre-existing attribute. ■ ■

M2 — Materialization is commutative. For each $r \in L$, LHS computes $e_1(r)$ and then $e_2(r)$, attaching both; RHS computes $e_2(r)$ and then $e_1(r)$. Because e_1 and e_2 depend only on r and not on

each other's intermediate result, the two sequences yield the same extension $r \oplus \{c_1 \mapsto e_1(r), c_2 \mapsto e_2(r)\}$. The multisets are equal. ■ ■

M3 — Materialization through projection. For $r \in L$, the LHS produces $\pi_A(r) \oplus \{c \mapsto e(\pi_A(r))\}$. The RHS produces $r \oplus \{c \mapsto e(r)\}$ (using the same e since e depends only on A), then projects to $(A \cup \{c\})$. By the support of e, $e(r) = e(\pi_A(r))$, and the resulting tuples agree. The freshness of c ensures the projection is well-formed. ■ ■

V1 — Vector selections on distinct columns commute. The composite predicate is $(\|V_a - q_a\| < \tau_a) \wedge (\|V_b - q_b\| < \tau_b)$, which is equivalent to $(\|V_b - q_b\| < \tau_b) \wedge (\|V_a - q_a\| < \tau_a)$. By A1, the selection-composition identity, the two orderings collapse to the same composite selection. ■ ■

J1 — Vector selection pushdown through join. Direct instantiation of A4 with the vector predicate as φ. The result is the same multiset because $\|V_a(r) - q\| < \tau$ is independent of any attribute of M. ■ ■

T1 — Limit after order equals top-K. By definition, $\tau_O(L)$ produces the rows of L in non-decreasing order of O, with stable tie-breaking on remaining columns; Λ_K then takes the first K. The result is precisely the K smallest rows under O. The two plans are logically equal. The interesting observation is *physical*: the LHS in the algebra may be implemented as `Sort` followed by `Limit`; the RHS as `TopKHeap` with incremental bounds. They are equivalent in result but very different in cost. ■ ■

G1 — Selection pushdown through graph expansion. Let R be the set of rows of L satisfying φ; the seed set of LHS is L, of RHS is R. Let $\text{reach}(g, d, S)$ denote the set of nodes reachable from a seed set S by rule g within depth d. Monotonicity of reachability gives $\text{reach}(g, d, R) \subseteq \text{reach}(g, d, L)$. The σ_φ on the LHS then removes any row whose origin row is not in R (since φ depends only on the origin's attributes), so the two multisets agree. ■ ■

A.2 Proof of RBO Confluence

RBO Confluence. Termination holds by the weight function $w(P) = \alpha s(P) + \beta r(P) + \gamma p(P)$ of Section 3.5: every rewrite rule's LHS is strictly larger than its RHS under w, so the rewriting system $\to_{\text{RBO}}$ has no infinite sequences. Local confluence holds by the critical-pair analysis of Section 3.5: every overlapping redex pair is joinable by a finite sequence of further rewrites (M1∩M1 joins by A1 then M2; M1∩J1 joins by J1 then M1; G1∩A4 joins by G1 by the monotonicity argument of the G1 proof; M2∩V1 joins by the confluent closure of M2, V1, and A1; T1∩M1 joins by M1 then T1). By Newman's lemma, termination plus local confluence implies confluence, and the normal form is unique. ■ ■

A.3 Proof of CBO complexity bound

CBO complexity bound. The CBO's search space is the product of the RBO's plan space and the per-operator physical-realization choice. From Section 4.11, the RBO's plan space is bounded by c^n for a constant c depending on the rule set and n operator occurrences. The CBO adds a factor of $|\text{Phys}|^n$ (at most 4 physical realizations per logical operator for the present inventory, but up to a small constant p in general). The product $|\text{Space}_{\text{RBO}}(Q)| \cdot |\text{Phys}|^n$ is exponential in n in the worst case. Standard pruning heuristics (Selinger-style dynamic programming for the $\bowtie$-fragment, greedy selectivity ordering for vector predicates, top-K pruning for $\tau\,\Lambda$) reduce the practical runtime to polynomial for the common subclasses. The bound remains exponential in the worst case; polynomial subclasses are characterized in Section 6.3. ■ ■

A.4 Reference tables

Physical operator	Cost vector (io, cpu, mem, approx)
$F(R)$ (file scan)	$(B(R),\ 0,\ B(R),\ 0)$
$B_{col,range}(R)$ (B-tree)	$(O(\log N + k),\ 0,\ k,\ 0)$
$H_{V,q,\tau}(R)$ (HNSW)	$(O(ef \log N),\ O(ef\, d),\ O(ef),\ 1 - \text{recall}(ef, M))$
$H_{V,q,\tau}(R)$ (IVF)	$(O(nprobe \log nlist),\ O(nprobe\, d),\ O(nprobe),\ 1 - \text{recall}(nprobe, nlist))$
$H_{V,q,\tau}(R)$ (scan)	$(O(\lvert R\rvert),\ O(\lvert R\rvert d),\ O(\lvert R\rvert),\ 0)$
$I_{col,term}$ (inverted)	$(O(p),\ O(p \cdot s),\ O(p),\ 0)$
$G_{g,d}$ (graph)	$(O(\lvert V\rvert + \lvert E\rvert),\ O(vis),\ O(vis),\ 0)$
$\text{Sort}(O)$ / $\text{TopK}(O, K)$	$(O(B \log B),\ O(B \log B),\ O(B),\ 0)$ / $(O(B \log K),\ O(B \log K),\ O(K),\ 0)$
$\text{Materialize}(e \to c)$	$(0,\ O(\lvert R\rvert \cdot \text{cost}(e)),\ O(\lvert R\rvert),\ 0)$
HashJoin / NLJ	$(B(R) + B(S),\ O(\lvert R\rvert + \lvert S\rvert),\ O(\min(\lvert R\rvert, \lvert S\rvert)),\ 0)$ / $(B(R)\lvert S\rvert,\ O(\lvert R\rvert\lvert S\rvert),\ 0,\ 0)$

Table 4. Symbolic cost vectors for core physical operators. $B(R)$: page count; N: cardinality; p: postings size; s: per-posting score cost; ef, M: HNSW parameters.

Logical operator	Realization (storage primitive)
$\sigma_{\lVert V-q\rVert<\tau}(R)$	$H_{V,q,\tau}(R)$ (HNSW/IVF/scan)
$\sigma_{\lVert V-q\rVert<\tau}(R)$ (no index)	$\mu_{\lVert V-q\rVert\to d}\, F(R);\ \sigma_{d<\tau}$ (linear scan)
$\sigma_{\text{BM25}(t,q)>\theta}(R)$	$I_{t,q,\theta}(R)$ (inverted index)
$\sigma_{A\in range}(R)$	$B_{A,range}(R)$ (B-tree)
$\Gamma_{g,d}(L)$	$G_{g,d,dir}(L)$ (graph traversal)
$\tau_O\, \Lambda_K(L)$	$\text{TopK}(O, K)(L)$ (bounded heap)
$\tau_O(L)$ (no Λ)	$\text{Sort}(O)(L)$ (full sort)
$\mu_{e\to c}(L)$	$\text{Materialize}(e \to c)(L)$
$L \bowtie_{\psi} M$	HashJoin / NLJ / MergeJoin
$F(R)$	$F(R)$ (sequential scan)

Table 5. Logical-to-physical mapping table (appendix copy, no duplicate label).